Vivante Programming: Dump Mechanisms with VGLite®

Document Revision 1.02

04 March 2022

Compatible with Vivante software

for VGLite driver release versions from 1.0.4

Legal Notices

COPYRIGHT INFORMATION

DISCLOSURE/RE-DISTRIBUTION LIMITATIONS

The information contained herein may be directly re-distributed or may be adapted for re-distribution by SoC vendors who have licensed the related Vivante core IP, with the understanding that re-distribution is limited to those customers of the SoC vendor who have active and valid NDA or licenses applicable for the SoC which contains the related Vivante core IP. Otherwise, the information contained herein is not to be used by or disclosed to the third parties without the express written permission of an officer of Vivante Corporation or VeriSilicon Holdings Co., Ltd.
(VeriSilicon Distribution **LEVEL B: CONFIDENTIAL – DISTRIBUTION RESTRICTED**).

Upon request VeriSilicon provides this document in Word format for adaptive inclusion in documentation intended for the Soc vendor's customers. VeriSilicon requests a pre-release copy of the adaptation for review and approval. In the adapted document please identify the technology and features described in this document using the Vivante trademark and include a reference to the copyright owner, for example: "This section describes the Vivante® GPU and contains copyright material disclosed with permission of VeriSilicon®, who has authorized re-distribution of this material restricted to those NDA partners and licensees of *[the SoC vendor]* who are engineering *[SoC vendor's product]* which includes the discussed Vivante IP."

TRADEMARK ACKNOWLEDGMENT

VeriSilicon® and the VeriSilicon logo design are the trademarks or the registered trademarks of VeriSilicon Holdings Co., Ltd. Vivante® is a registered trademark of Vivante Corporation. All other brand and product names may be trademarks of their respective companies.

For our current distributors, sales offices, design resource centers, and product information, visit our web page located at http://www.verisilicon.com.

For technical support, please email vivante-support@verisilicon.com.

Table of Contents

1 Overview

This document describes simple methods for dumping information about hardware register contents and command buffer values with VGLite software.

1.1 Applicable Hardware and Software

This mechanism is a stable component of Vivante VGLite driver software from the 1.0.4 and later releases and is compatible only with Vivante VGLite IP cores. The mechanisms described in this document are not applicable for the Unified GCCORE drivers.

1.2 Additional Resources

This document assumes the user has Vivante VGLite driver and test source software and all documentation provided with the software release.

1.3 VGLite Dump Mechanism Overview

When detecting a Vivante VG core hang, the Vivante VGLite software can dump different levels of VG IP information by selecting the appropriate dump instruction. Information that can be dumped includes the following:

- **Current values of VG debug registers**
  - These values expose some important VG internal status information useful to the VG hardware designer for diagnosing the issue.
- **Content of the current command buffer**
  - The VGLite command buffer can also be used to analyze VG IP behavior.
- **Additional driver internal information is not available**
  - The VGLite driver is intended to be very "Lite." No extra information can be provided.

2 Building dump_info

Vivante's VGLite dump_info is a test application which reads register values in compatible VG cores. This section describes how to build the application for your custom environment. The instructions below assume dump_info is part of the test package and that all tests in the **samples** directory will be built at the same time.

2.1 Prerequisties

This instruction assumes the following additional files are available in the indicated directories:

```
\$VIVANTE_SDK_DIR/drivers/libvg_lite.so
\$VIVANTE_SDK_DIR/inc/vg_lite.h
\$VIVANTE_SDK_DIR/inc/vg_lite_error.h
```

2.2 Build the dump_info application

Step 1. Expand the **VIVANTE_VGLite_Src_tst_*.tgz** package. (If the tool is provided as a single application package, adapt these instructions as needed.)

Step 2. Review the **README** file which contains instruction on building the test samples. It may contain updated information not included in this document.

Step 3. Edit the **./test_build_sample.sh** file to provide custom definitions for the environment variables needed for **dump_info**:

export VIVANTE_SDK_DIR specify the location of the Vivante VGLite driver SDK where the binary will be installed

export CROSS_COMPILE<CROSS_COMPILER_PREFIX> specify the correct cross-compile toolchain for your build environment

Step 4. Build **dump_info** by running the following script:

```
/test_build_sample.sh
```

Step 5. The binary **dump_info** will be installed to **$VIVANTE_SDK_DIR/samples/dump_info**

3 Run dump_info to Dump Register Info

The application dump_info is run at the Linux command line.

Syntax:

At the command line, either navigate to the directory where **dump_info** resides or type the path and application name. For example,

```
root@myplatform:/home/sdk/drivers# ./dump_info
```

There are currently no command line options needed and no help detail for this simple application.

A prompt will appear asking you to:

```
please input register address:
```

Input a hexadecimal value in the format 0xn, such as 0x1, 0xc, etc. This value will be the AHB Byte address of the register whose address you want to dump. Refer to the Vivante AHB Register Specification document for register descriptions. The valid range for VGLite cores is usually 0x0 to 0x1FF and 0xA00 to 0xA7F.

```
0x4
```

The dump_info application will respond with a line similar to:

```
result is 0x7fffffff
```

This value will represent the current register reset value for this register. If the input number is not a valid register address, the returned result is 0.

Repeat this sequence for each register address you wish to query.

For example:

For	Input	GCNanoLiteV returns	GCNanoUltraV returns	GC355 returns
GCChipId	0x20	0x00000255	0x00000265	0x00000355

3.1 Interpreting dump_info output

A description of select registers accessible via AHB is provided in the document Vivante AHB Accessible Registers compatible with your VG IP. Vivante tries to maintain a consistent register definition scheme across its cores. For this reason, some field bits may have a field name defined, but the field may not be in use for that specific core revision. For example, a field may be obsolete, or a field defined for use in 2D cores would not be used in a VG or 3D core. A reset value specified for such fields may be ignored.

3.1.1 Exposed Register Checklist:

These registers are the AHB registers typically exposed for VGLite –compatible cores.

AHB Module	Exposed Register	AHB Byte Address	GPU DWord Address	Read/ Write	Observed Register Value
HI	AQHiClockControl	0x000	0x0000	R/W	
HI	AQHiIdle	0x004	0x0001	R	
HI	AQAxiConfig	0x008	0x0002	R/W	
HI	AQAxiStatus	0x00C	0x0003	R	
HI	AQIntrAcknowledge	0x010	0x0004	R	
HI	AQIntrEnbl	0x014	0x0005	R/W	
HI	GCChipId	0x020	0x0008	R	
HI	GCChipRev	0x024	0x0009	R	
HI	GCChipDate	0x028	0x000A	R	
HI	GCChipTime	0x02C	0x000B	R	
HI	gcregHIChipPatchRev	0x098	0x0026	R/W	
PM	gcModulePowerControls	0x100	0x0040	R/W	
MC	AQMemoryDebug	0x414	0x0105	R/W	
MC	AQMemoryFE	0x41C	0x0107	R/W	
MC	AQRegisterTimingControl	0x42C	0x010B	R/W	
FE	gcregFetchAddress	0x500	0x0140	W	
FE	gcregFetchControl	0x504	0x0141	W	

4 Dumping the Command Buffer

4.1 Activate the Command Buffer Dump Mechanism

Use the following steps to dump VGLite command buffer information:

Step 1. In VGLite/vg_lite.c enable the macro which activates the command buffer dump, by setting the switch for macro **DUMP_COMMAND** to **1**:

```
#define DUMP_COMMAND 1
```

Step 2. Rebuild the VGLite driver package to generate all new driver executables. Refer to README file in VGLite for build instruction detail.

Step 3. Run the desired application as usual. A text named **`Commandbuffer_pidxxx.txt`** will be generated in the application's output directory.

4.2 Output Format

Each text line of the output will usually consist of two elements. The first element contains the the hardware command opcode expressed as a hexadecimal number in the last two digits. The second item will be the data for that command. In the following content is an example:

```
Command buffer: 0x30010a00, 0x00000001,
Command buffer: 0x30010a02, 0xffff0000,
Command buffer: 0x30010a10, 0x00010003,
Command buffer: 0x30010a11, 0x00001000,
Command buffer: 0x30010a12, 0x00000a00,
```

Output format may vary by command type. The following are the VGLite commands. Note that the command codes for VG are different than those for other Vivante cores. The output data for each command will vary by opcode.

VG Command	OPCODE Numeric	Second item Data
END	0x00	interrupt
SEMAPHORE	0x01	id
STALL	0x02	id
STATE	0x03	address
DATA	0x04	count
WAIT	0x05	not supported
CALL	0x06	count
RETURN	0x07	supported, no data
NOP	0x08	not supported
RESTART	0x09	not supported

Document Revision History

Document Revision	Date	Compatible HW rev	Notes
1.02	2022-03-04	as previous Vivante VGLite Software (from 1.0.4 of 2015-11)	Filename modified. Legal Notices, General: Update branding layout to include VeriSilicon. Miscellaneous refinements.
1.01	2017-01-12	as previous Vivante VGLite Software (from 1.0.4 of 2015-11)	Update color scheme
1.00	2015-11-13	Vivante VGLite Software 1.0.x series (from 1.0.4 of 2015-11)	